Lecture 8 (Part 2) — Exception Handling

- Why exception handling is needed
- try / except / else / finally
- Custom exceptions
- Business use cases

Why Do We Need Exception Handling?

- Programs crash without handling errors.
- User input is unpredictable (e.g., letters instead of numbers).
- Files and networks may fail.
- Business systems require reliability.
- Exception handling prevents system breakdown.

What is an Exception?

- An exception is a runtime error that stops program execution.
- Without handling $\rightarrow$ program crashes.
- With handling $\rightarrow$ program continues safely.

Basic try/except Syntax

- try block contains risky code.
- except block handles specific errors.

```
try:
    num = int(input("Enter number: "))
except ValueError:
    print("Invalid input! Please enter digits only.")
```

Multiple Except Blocks

- Different errors need different handling.
- Improves reliability and debugging.

```
try:
    result = 10 / x
except ZeroDivisionError:
    print("Cannot divide by zero.")
except TypeError:
    print("Wrong type used.")
```

else and finally Blocks

- else runs only when no error occurs.
- finally always runs (cleanup operations).

```
try:
    f = open("data.txt")
except FileNotFoundError:
    print("File missing.")
else:
    print("File opened successfully.")
finally:
    print("Done!")
```

Creating Custom Exceptions

- Used for business-specific errors.
- Makes code cleaner and more meaningful.

```
class LowBalanceError(Exception):
    pass

def withdraw(balance, amount):
    if amount > balance:
        raise LowBalanceError("Insufficient funds!")
    return balance - amount
```

Business Example — Safe Banking Withdrawal

- Combine custom exception with try/except.

```
try:
    new_balance = withdraw(5000, 7000)
except LowBalanceError as e:
    print("Transaction failed:", e)
else:
    print("Withdrawal successful. New balance:", new_balance)
```

Practice Tasks (Part 2)

- Create a calculator that handles invalid input + division errors.
- Create login system that handles wrong password, user not found.
- Implement file-safe reader using try/except/finally.

Summary — Exception Handling

- try/except prevents crashes.
- else and finally improve control flow.
- Custom exceptions make business rules clear.
- Reliable systems must handle exceptions properly.